

Master 1 Calcul Haute Performance, Simulation

Rapport de projet

Méthodes Ensemblistes pour la Modélisation

Auteurs :

GUERROUDJ Djohra
LAKHANI Naïla
OUNNAS Masten
ZAMOUM Tassadit

Encadrant :

M. Mathys JAM

16 Décembre 2025

Table des matières

1 Introduction **4**

2 Étude de cas : Le jeu de données Iris **4**
2.1 La base de données . . . 4
2.2 Définition et Principe [3] . . . 5
2.3 Les hyperparamètres . . . 5
2.3.1 Les objectifs . . . 5
2.3.2 La profondeur . . . 6
2.3.3 Le gain . . . 6
2.3.4 Le nombre d'échantillons . . . 6
2.3.5 La règle d'arrêt . . . 6
2.4 L'Entraînement . . . 7
2.5 Validation . . . 8

3 Implémentation de l'arbre **8**
3.1 Données et prétraitement . . . 8
3.1.1 15k_ga_adaptive.csv. . . . 8
3.1.2 15k_hvs.csv. . . . 9
3.1.3 15k_random.csv. . . . 9
3.1.4 30k_ga_adaptive.csv. . . . 9

4 Analyse **10**
4.1 Tableaux . . . 10
4.2 Histogrammes . . . 10
4.3 Interprétation . . . 11
4.4 Discussion . . . 11

5 Analyse de la Vectorisation (SIMD) **11**
5.1 Inspection du Code Assembleur . . . 11

6 Optimisation **12**
6.1 Optimisation Algorithmique : Formule de Koenig-Huygens . . . 12

7 Approche Ensembliste : Bagging et Boosting **12**
7.1 Le Bagging (Bootstrap Aggregating) . . . 12
7.1.1 Concept Théorique . . . 12
7.1.2 Implémentation et Choix des Hyperparamètres . . . 13
7.2 Le Gradient Boosting . . . 13
7.2.1 Concept Théorique . . . 13
7.2.2 Lien Mathématique : Gradient et Résidu [9] . . . 13
7.2.3 Algorithme Implémenté . . . 13

8 Étude Paramétrique **13**
8.1 Précision . . . 14
8.2 Analyse du Coût Computationnel et Scaling . . . 16

9 Étude d'Optimisation Bas-Niveau : Impact du Compilateur **17**
9.1 Protocole Expérimental . . . 17

10 Analyse d'Optimisation Bas-Niveau et Memory Wall **17**
10.1 Analyse par Jeu de Données . . . 17

11 Conclusion **20**

Environnement Expérimental

Cette section détaille les spécifications matérielles et logicielles utilisées. La connaissance précise de l'architecture est indispensable pour interpréter les performances, notamment l'impact du cache L3 et des jeux d'instructions vectoriels.

Environnement Expérimental

Matériel (Hardware)

Les tests sont exécutés sur une station mobile **ASUS TUF Gaming A16 Advantage Edition (2023)**, basée sur une architecture AMD complète.

- **Processeur (CPU) :** AMD Ryzen™ 7 7735HS.
  - **Micro-architecture :** x86 64 / Zen 3+ ("Rembrandt R"), gravure 6nm FinFET.
  - **Topologie :** 8 Cœurs Physiques / 16 Threads Logiques (SMT).
  - **Fréquence :** Base 3.2 GHz / Boost 4.75 GHz.
  - **Hiérarchie Cache (Extraite via hwloc) :**
    - *L1-Instruction :* 8×32 KB (8-way associative).
    - *L1-Data :* 8×32 KB (8-way associative).
    - *L2 Cache :* 8×512 KB (Unified, Private per core).
    - *L3 Cache :* **16 MB** (Unified, Shared by all cores).
- **Mémoire (RAM) :** 16 GB DDR5-4800 MHz (SO-DIMM).
  - Architecture NUMA : Single Node (Uniform Memory Access).
- **Stockage :** SSD NVMe 512 GB (PCIe Gen 4).

Logiciel (Software)

L'environnement est configuré pour minimiser le bruit système ('OS jitter') et garantir l'utilisation des extensions ISA récentes.

- **OS :** Linux Mint 22.2 "Zara" (Base Ubuntu 24.04 LTS "Noble").
- **Kernel :** Linux 6.14.0-29-generic.
- **Compilateur :** GCC 13.3.0 (Target : x86_64-linux-gnu).
- **Configuration Build (CMake 3.14+) :**
  - *Standard :* C++17.
  - *Flags :* `-O3 -march=native -Wall -Wextra`.
  - *Vectorisation :* Activation implicite des instructions AVX2, FMA3, BMI1/BMI2 supportées par Zen 3+.
- **Validation & Analyse :** Google Test (GTest) 1.10+, Python 3.12 (Matplotlib/NumPy).

Spécification du projet

Objectif du projet

Le projet a pour objectif la mise en œuvre et l'évaluation de différents modèles de régression basés sur les **arbres de décision**, ainsi que sur des méthodes d'ensemble telles que le **Bagging** et le **Boosting**. Pour atteindre ces objectifs, plusieurs technologies ont été utilisées, chacune choisie pour répondre à un besoin spécifique du projet.

Choix du langage : C++

Le cœur de l'implémentation a été réalisé en **C++**, principalement pour des raisons de performance et de contrôle précis de la mémoire. Le C++ permet une exécution rapide, ce qui est essentiel pour manipuler des jeux de données volumineux et construire des arbres de décision efficacement. L'utilisation de pointeurs intelligents tels que `std::unique_ptr` et de structures légères optimise l'allocation mémoire, ce qui est particulièrement important pour les méthodes d'ensemble que l'on souhaite implémenter. De plus, le C++ s'intègre facilement dans des projets modulaires utilisant **CMake**, facilitant ainsi la compilation et la gestion des dépendances.

Tests unitaires : Google Test (GTest)

Pour garantir la robustesse de l'implémentation, nous avons utilisé **Google Test (GTest)** afin d'écrire des tests unitaires. Cela permet d'organiser les tests en suites et cas de test dédiés à chaque module, comme le `DataLoader`, le `DecisionTree`, le `BaggingRegressor` et le `BoostingRegressor`. L'utilisation d'assertions spécialisées telles que `EXPECT_EQ`, `EXPECT_NEAR` ou `EXPECT_FALSE` permet de vérifier automatiquement les résultats sans arrêter l'exécution des autres tests. Ce choix améliore également la maintenabilité du projet, car les tests peuvent être étendus facilement pour couvrir de nouveaux cas ou modules sans modifier les anciens tests.

Analyse et visualisation : Python

Bien que l'implémentation principale soit en C++, **Python** a été utilisé pour la création de graphe et l'analyse de nos résultats. Python facilite le traitement et la visualisation des métriques telles que le RMSE, le MAE et le temps d'exécution grâce à des bibliothèques comme `pandas` et `matplotlib`. La bibliothèque `scikit-learn`[1] a également été utilisée afin de mieux comprendre le fonctionnement des algorithmes de machine learning.

Compilation et gestion du projet : CMake

CMake a été choisi comme outil de compilation et de gestion du projet. Il facilite la gestion des dépendances, notamment pour l'intégration de GTest. CMake automatise la génération des exécutables et organise proprement les fichiers sources et headers, ce qui rend le projet facilement maintenable et évolutif.

1 Introduction

Dans l'analyse des **performances** d'algorithmes de calcul scientifique, il est fréquent d'étudier l'impact de plusieurs **paramètres** d'exécution (taille des données, organisation mémoire, parallélisme, options algorithmiques, etc.) sur le temps d'exécution d'un programme. Lorsque le nombre de **paramètres** augmente, l'évaluation exhaustive de toutes les configurations possibles devient rapidement irréalisable, en raison du coût élevé des mesures et du caractère combinatoire de l'espace des paramètres [2].

Une approche consiste alors à exploiter un ensemble limité de mesures afin de construire un **modèle de régression** reliant les paramètres de configuration au temps d'exécution observé. Un tel modèle permet de réduire le nombre d'exécutions nécessaires tout en conservant une estimation fiable du comportement du programme.

Dans ce projet, nous nous appuyons sur un jeu de données regroupant des exécutions d'un programme de **factorisation LU** (DGETRF) sous différentes configurations. La **variable cible** correspond explicitement au temps d'exécution mesuré, tandis que les **variables explicatives** décrivent les paramètres de configuration. L'objectif est d'implémenter, à partir de zéro en langage C++, des techniques ensemblistes pour la régression.

Nous commençons par l'implémentation d'un **arbre de décision** pour la régression, utilisé comme modèle de base. Les premières expérimentations montrent que ce modèle peut fournir des prédictions satisfaisantes, mais qu'il est sensible à la présence de valeurs extrêmes dans les données. Dans ce cas, de faibles variations du jeu d'apprentissage peuvent conduire à des structures d'arbres différentes, et donc à des prédictions moins stables, ce qui met en évidence une variance élevée du modèle.

Afin de limiter cette instabilité, nous étudions ensuite deux méthodes ensemblistes classiques :le **Bagging** et le **Boosting**. Ces méthodes reposent sur la **combinaison** de plusieurs arbres de régression dans le but d'améliorer la stabilité des prédictions. Le **Bagging** vise principalement à réduire la variance par moyennage de modèles, tandis que le **Boosting** construit les modèles de manière séquentielle en cherchant à corriger progressivement les erreurs précédentes.

Ce travail a pour objectif de comparer le comportement d'un arbre de régression seul et de méthodes ensemblistes, en étudiant l'impact du choix du modèle, du critère de division et des paramètres d'apprentissage sur la qualité des prédictions et le coût de calcul associé.

2 Étude de cas : Le jeu de données Iris

2.1 La base de données

On s'intéresse dans un premier temps à un example minimal : le jeu de données Iris fourni par la bibliothèque Python `scikit-learn`. Cette bibliothèque propose des fonctions permettant de construire des arbres de décision à partir de jeux de données, de prédire à partir de ses arbres ainsi que d'autres fonctions liées au machine learning. L'objectif est donc d'exploiter ces outils afin de comprendre leur fonctionnement et, par la suite, de concevoir nous-mêmes un algorithme reproduisant des résultats similaires.

Le jeu de données **Iris** est constitué de fleurs classées selon la longueur et la largeur de leurs pétales et de leurs sépales. Chaque fleur appartient à l'une des trois classes suivantes : **Setosa**, **Versicolor** ou **Virginica**. La base de données brute se présente sous la forme suivante :

Sepal length (cm)	Sepal width (cm)	Petal length (cm)	Petal width (cm)	Target	Target name
5.1	3.5	1.4	0.2	0	setosa
4.9	3.0	1.4	0.2	0	setosa
4.7	3.2	1.3	0.2	0	setosa
4.6	3.1	1.5	0.2	0	setosa
5.0	3.6	1.4	0.2	0	setosa
⋮	⋮	⋮	⋮	⋮	⋮
6.7	3.0	5.2	2.3	2	virginica
6.3	2.5	5.0	1.9	2	virginica
6.5	3.0	5.2	2.0	2	virginica
6.2	3.4	5.4	2.3	2	virginica
5.9	3.0	5.1	1.8	2	virginica

TABLE 1 – Extrait du jeu de données Iris

Il est également possible de représenter ces données sous forme de graphiques afin d'en faciliter la lecture et l'interprétation :

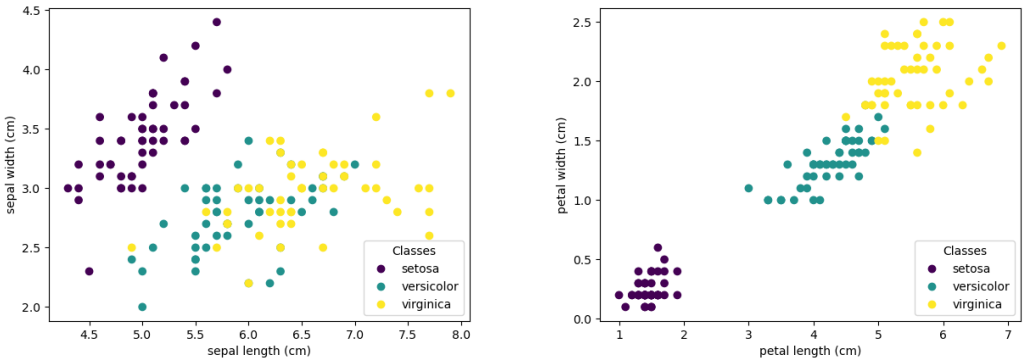


FIGURE 1 – Distribution des fleurs en fonctions de leurs dimensions

L'objectif est de prédire la nature d'une fleur uniquement à partir de ses dimensions. Il s'agit donc d'exploiter les données existantes et de les partitionner afin de mettre en évidence une relation entre les **paramètres** (dimensions des pétales et des sépales) et l'**étiquette** associée (la classe de la fleur). Il n'est pas possible d'établir cette relation à l'aide d'une simple équation linéaire. Il est donc nécessaire de recourir à un modèle non linéaire, tel qu'un **arbre de décision**.

2.2 Définition et Principe [3]

Un **arbre de décision** (dans notre cas, un arbre de régression) est un modèle prédictif qui partitionne récursivement l'espace des **échantillons** en sous-espaces distincts. L'objectif est de grouper les échantillons ayant des valeurs cibles similaires. L'arbre de décision est construit à partir de **tests binaires** successifs. À chaque nœud, chaque échantillon est soumis à une condition. Si celle-ci est vérifiée (TRUE), l'échantillon est dirigé vers le nœud gauche, sinon, il est dirigé vers le nœud droit. La prédiction pour une nouvelle donnée x est obtenue en descendant l'arbre jusqu'à une feuille. La valeur prédite $\hat{y}$ est alors la moyenne des valeurs cibles des échantillons d'entraînement présents dans cette feuille.

2.3 Les hyperparamètres

Les **hyperparamètres** permettent d'optimiser la construction d'un arbre en utilisant leurs valeurs comme critère d'arrêt. Penchons nous sur leur cas :

2.3.1 Les objectifs

Dans un arbre de régression **l'objectif**, ou le critère de division, détermine comment les données sont séparées à chaque nœud. Concrètement, pour chaque variable j et pour plusieurs seuils candidats s, on évalue la qualité de la séparation induite. En voici quelques examples.

MSE (Mean Squared Error)[4] Pour un nœud contenant un ensemble D d'indices, et une cible y, le MSE est :

$$\text{MSE}(D) = \frac{1}{|D|}\sum_{i\in D}(y_i - \bar{y}_D)^2, \quad \text{où } \bar{y}_D = \frac{1}{|D|}\sum_{i\in D} y_i. \tag{1}$$

Le **MSE** pénalise fortement les erreurs importantes (car elles sont au carré), ce qui tend à favoriser des splits qui réduisent la variance intra-nœud. C'est cet objectif que nous avons décidé de retenir pour la construction de notre algorithme.

RMSE (Root Mean Squared Error) Le **RMSE** est la racine du MSE :

$$\text{RMSE}(D) = \sqrt{\text{MSE}(D)}. \tag{2}$$

Il est exprimé dans la même unité que y, ce qui facilite parfois l'interprétation. Utilisé comme critère de *split*, il conserve la sensibilité du MSE aux erreurs extrêmes, tout en modifiant l'échelle de la perte.

MAE (Mean Absolute Error)[4] Le **MAE** est défini par :

$$\mathrm{MAE}(D) = \frac{1}{|D|} \sum_{i \in D} |y_i - \tilde{y}_D| \,, \tag{3}$$

Théoriquement, $\tilde{y}_D$ est la médiane car elle minimise le MAE.

L'indice GINI [3] Avec p_i la probabilité de tomber sur l'élément i lorsque l'on tire aléatoirement dans un nœud de l'arbre, **l'indice de Gini** est défini par :

$$\mathrm{Gini} = 1 - \sum_{i=1}^{n} p_i^2$$

Plus la valeur de l'indice est proche de zéro, plus le nœud est pur.

2.3.2 La profondeur

La **profondeur** d'un arbre correspond au nombre de "niveaux" que l'arbre possède, la profondeur maximale d'un arbre de N échantillons est égale à :

$$\mathrm{Max_depth} = \log_2(N)$$

Cependant, avoir une profondeur maximale n'est pas optimal car cela mène à du **surapprentissage**[5]. L'arbre construit est certes, plus précis mais ne prédit pas correctement des données nouvelles, non utilisés pour la construction : l'arbre est trop "spécifique".

Dans notre cas, nous allons expérimenter avec différentes valeurs de profondeur afin d'analyser l'impact de cet **hyperparamètre** sur l'abre. Ayant des jeu de données de 15000 et 30000 échantillons, il n'est pas intéressant d'aller plus loin que des profondeurs égales à 15 ($\log_2(15000) \approx 14$ et $\log_2(30000) \approx 15$).

2.3.3 Le gain

Le **gain** est le critère utilisé pour déterminer si le split effectué est efficace ou non. On peut calculer ce gain en utilisant les formules utilisées lors de l'implémentation de l'objectif. `scikit-learn` utilise **l'indice de Gini** pour déterminer la pureté de ses noeuds[6]. Nous avons décidé de continuer avec le **MSE** pour calculer notre gain :

$$\mathrm{Gain} = MSE(parent_node) - MSE(child_node)$$

Le but étant donc au fur et à mesure de la construction d'obtenir un **gain** de plus en plus grand.

2.3.4 Le nombre d'échantillons

Comme son nom l'indique, c'est le nombre d'échantillons que possède chaque noeud. On peut également compter le nombre d'échantillons qu'on aura obtenu après un split dans chaque noeud fils gauche et droite.

2.3.5 La règle d'arrêt

Tous ces **hyperparamètres** peuvent servir de condition d'arrêt pour le processus de construction. Pour éviter le sous-apprentissage ou le surapprentissage[5], il faut déterminer quelle configuration est **optimale**. On peut choisir de finir la construction lorsqu'une certaine profondeur maximale, un certain gain minimum ou encore un certain nombre d'échantilllons minimum est atteint. Dans le cas `scikit-learn`, la règle d'arrêt dépend du gain[6]. L'arbre s'arrête lorsque tous les noeuds possèdent un indice de Gini égal à zéro. Dans notre implémentation, nous avons décidé de définir une valeur minimale d'échantillons dans chaque noeud ou bien une valeur maximale de profondeur à atteindre.

2.4 L'Entraînement

La première étape de la construction d'un arbre de décision est la phase **d'entraînement**. Durant cette phase, les données préalablement collectées sont triées selon différents critères. À chaque **nœud** de l'arbre, un **seuil** est appliqué. Le seuil est choisi à l'aide des hyperparamètres définis plus tôt, l'algorithme choisit le seuil optimal. Dans le cas de `scikit-learn`, le seuil est la valeur qui produit un indice de Gini minimal[6] car c'est la valeur par défaut de la bibliothèque pour un arbre de classification. Dans notre implémentation qui sera un arbre régressif, nous allons utiliser le calcul du gain à l'aide du MSE.

Le terme "**samples**" indique le nombre de données, c'est-à-dire le nombre de fleurs, présentes dans chaque nœud de l'arbre. Le tableau x représente les paramètres d'entrée :

1. `x[0]` : longueur du sépale
2. `x[1]` : largeur du sépale
3. `x[2]` : longueur du pétale
4. `x[3]` : largeur du pétale

Le tableau **value** représente l'effectif de chaque classe dans un nœud donné :

1. `value[0]` : nombre de fleurs Setosa
2. `value[1]` : nombre de fleurs Versicolor
3. `value[2]` : nombre de fleurs Virginica

Au niveau du premier nœud (la racine), la condition est

$$x[3] \leq 0.8 \tag{4}$$

Cela signifie que si la largeur du pétale de la fleur est inférieure ou égale à `0.8`, elle est orientée vers la branche gauche, dans le cas contraire, elle est orientée vers la branche droite. Ce processus se poursuit jusqu'à ce que la condition d'arrêt définie est remplie, ici lorsque **l'indice de Gini** est nul.

Voici ce qui se produit au premier niveau de profondeur de l'arbre :

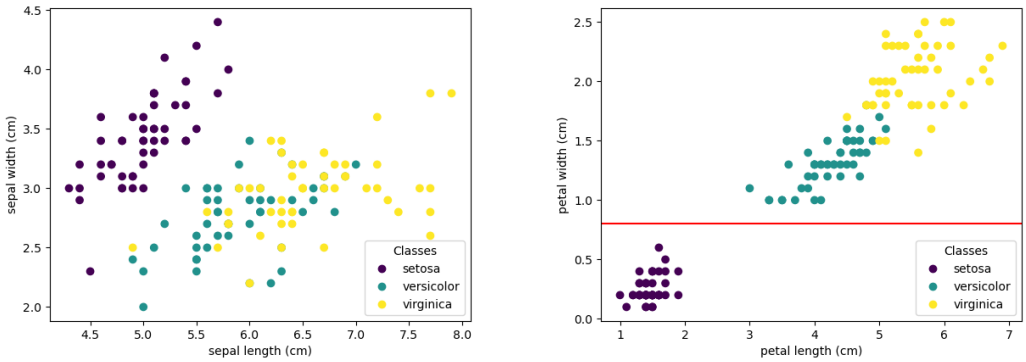


FIGURE 2 – Split à la condition $x[3] \leq 0.8$

Le split se reproduit pour chaque condition, on obtient finalement les représentations suivantes :

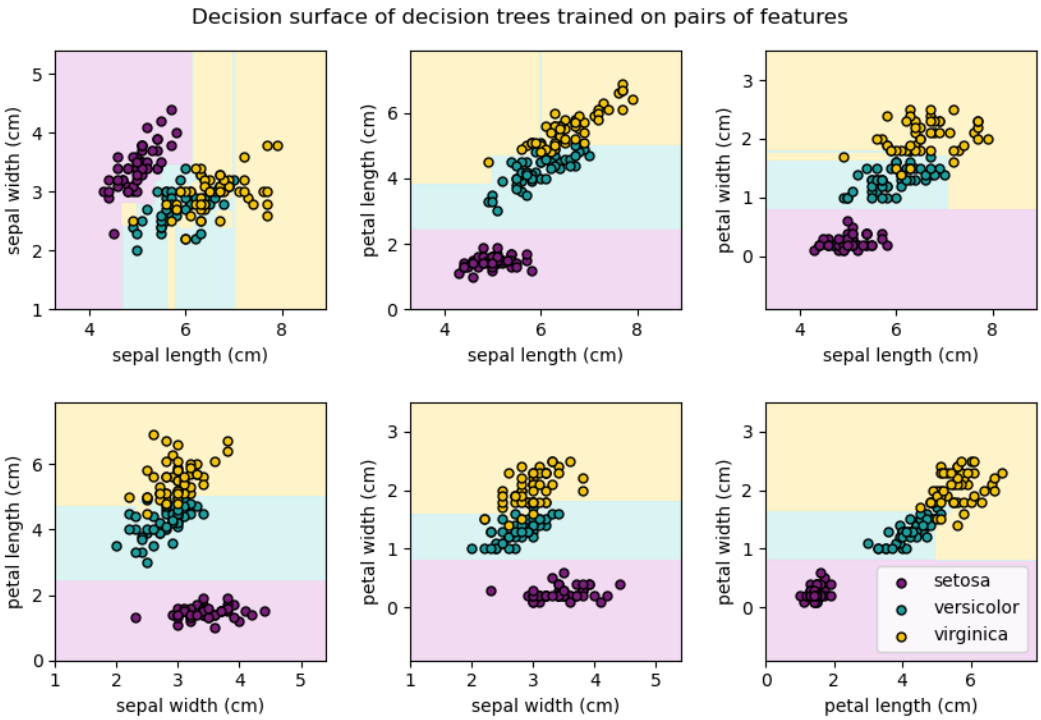


FIGURE 3 – Résultat du partitionnement de l'arbre entraîné

Ces graphiques montrent le partitionnement des données selon différentes projections 2D. Chaque fleur est progressivement isolée en fonction de sa classe. L'étape **d'entraînement** de l'arbre est alors achevée.

2.5 Validation

Il reste ensuite à tester l'arbre afin de vérifier la qualité du partitionnement effectué. La phase de test consiste à fournir à l'arbre des données dont la classe est connue. Les échantillons sont propagés dans l'arbre en appliquant les règles de split. Cette prédiction est ensuite comparée à la classe réelle afin d'estimer l'erreur du modèle. Idéalement, le jeu de données est préalablement séparé aléatoirement en deux parties, par exemple selon une **répartition 80/20**, afin de disposer d'un ensemble de test indépendant de l'ensemble d'entraînement. La bibliothèque `scikit-learn`, ainsi que le jeu de données Iris, constituent une excellente introduction aux arbres de décision. L'example précédent détallait un problème de classification, on s'intéresse désormais à un problème de régression.

3 Implémentation de l'arbre

3.1 Données et prétraitement

Comme pour l'étude de cas, commençons par analyser les données mis à notre disposition. Pour rappel, nous nous intéressons aux mesures de performance d'un algorithme de **factorisation LU** dans le but de pouvoir l'optimiser en déterminant quelles configurations nous donne les meilleurs performances. On possède quatre fichiers de jeu de données :

3.1.1 15k_ga_adaptive.csv.

Contient **15 000** échantillons générés par un algorithme génétique adaptatif. Les données suivent un pattern structuré, permettant aux modèles d'apprendre facilement et de produire de bonnes prédictions avec peu de bruit.

3.1.2 15k_hvs.csv.

Contient **15 000** échantillons avec beaucoup de bruit et des valeurs aberrantes (y = 30). Ce fichier teste la robustesse des modèles et montre l'importance du filtrage des valeurs aberrantes pour réduire le MSE.

3.1.3 15k_random.csv.

Contient **15 000** échantillons générés aléatoirement, sans pattern prévisible. Ce jeu de donnée permet de mesurer la capacité des modèles à gérer des données imprévisibles et de comparer la performance sur des données non structurées.

3.1.4 30k_ga_adaptive.csv.

Contient **30 000** échantillons générés de la même manière que `15k_ga_adaptive.csv`. Les données restent structurées et peu bruyantes, mais le jeu de donnée plus grand permet de tester la scalabilité et le temps d'entraînement des modèles.

Afin d'illustrer ces jeux de données, voici un extrait du premier ficher `15k_ga_adaptive` :

p1	p2	p3	p4	p5	p6	p7	p8	matrix_x	matrix_y	performance
213	43	27	30	18	8	5	53	4784	4974	0.130534
29	108	4	3	22	3	92	2	2427	3109	0.0321952
15	94	10	21	11	9	70	98	3426	2805	0.0374015
230	153	24	11	22	20	32	18	4756	2779	0.0576987
197	249	31	13	22	30	66	19	1328	3868	0.0209786
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
32	8	2	0	8	1	98	1	3848	3075	0.0313442
46	82	3	22	20	6	94	61	4209	4097	0.0522107
26	0	1	14	1	3	98	1	2948	1924	0.0156462
46	82	4	22	20	3	94	61	4530	3965	0.049332
20	0	2	23	8	32	99	4	3985	1358	0.01555

Chacun de ces fichiers contient 11 colonnes :

• Huit colonnes qui représentent différents **leviers d'optimisation** du noyau, dont les noms ont étés délibérémment obfusqués :
- **p1** : [8, 256]
- **p2** : [0, 256]
- **p3** : [1, 32]
- **p4** : [0, 32]
- **p5** : [1, 32]
- **p6** : [0, 32]
- **p7** : [1, 99]
- **p8** : [1, 99]

• Deux colonnes représentant la dimension de la matrice (longueur et largeur)

• Une colonne représentant la **variable cible**, c'est-à-dire le temps d'exécution du programme.

On nous indique que le **bruit** de mesure est estimé à environ 10%, on remarque rapidement des valeurs abberantes présentes dans les jeux de données. Lors de certaines exécutions, la mesure de la performance plafonne à 30, ce qui correspond à des erreurs d'exécution. Ces valeurs peuvent fausser l'entraînement de l'arbre de décision et dégrader la qualité des prédictions. Pour remédier à cela, nous avons appliqué un filtrage directement lors du chargement des fichiers CSV afin d'éliminer ces valeurs aberrantes. Analysons l'impact de ce filtrage sur la précision et la performance de l'algorithme.

4 Analyse

On définit les hyperparamètres suivants :

- $Max_depth = 15$
- $Min_samples_split = 10$
- $Min_gain = 1e-7$

4.1 Tableaux

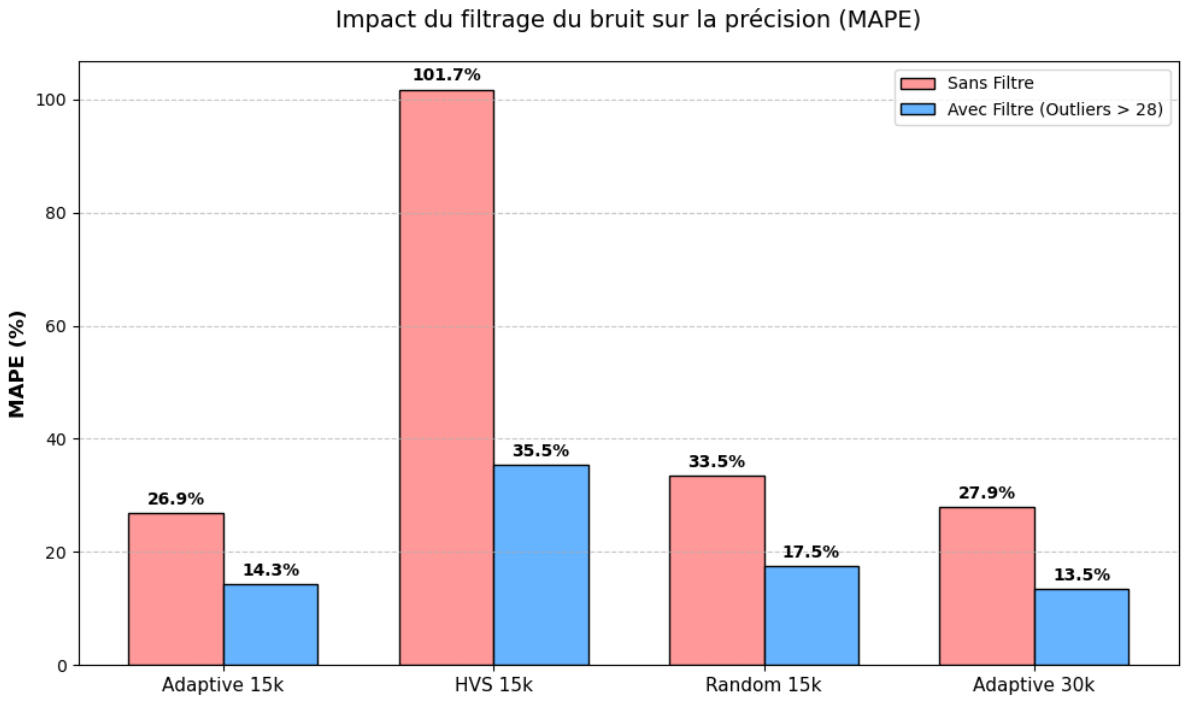
Dataset	Échantillons	RMSE	MAPE	Temps (s)
15k_ga_adaptive.csv	15 000	0.641224	26.8548 %	0.24574
15k_hvs.csv	15 000	3.29561	101.733 %	0.070073
15k_random.csv	15 000	0.725242	33.4992 %	0.0798762
30k_ga_adaptive.csv	30 000	0.585246	27.8935 %	0.169456

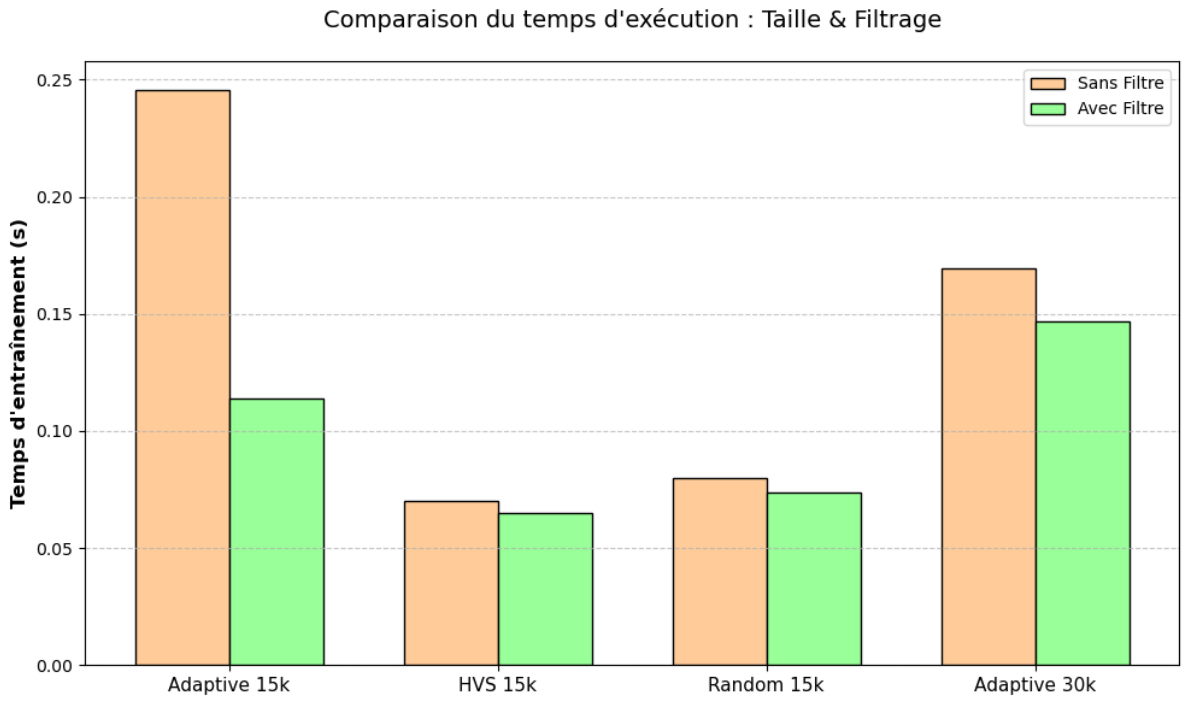
TABLE 2 – Résultats des arbres pour chaque jeu de donnée avant filtrage

Dataset	Échantillons	RMSE	MAPE	Temps (s)
15k_ga_adaptive.csv	14 994	0.0262656	14.2918 %	0.11398
15k_hvs.csv	14 745	0.0971736	35.4781 %	0.0649533
15k_random.csv	14 992	0.0355072	17.5441 %	0.0738087
30k_ga_adaptive.csv	29 988	0.0229475	13.46 %	0.146579

TABLE 3 – Résultats des arbres pour chaque jeu de donnée après filtrage

4.2 Histogrammes





4.3 Interprétation

Les performances sont nettement améliorées après suppression des valeurs aberrantes. La présence de valeurs égales à 30 génère des erreurs très importantes, notamment sur le fichier `15k_hvs.csv`. Le filtrage appliqué directement lors du chargement des données permet à l'arbre de rester stable et de réduire le **RMSE** sur l'ensemble de test. Cependant, le temps d'entraînement est normalement proportionnel à la taille du jeu de données et au filtrage : moins d'échantillons signifie un arbre plus rapide à construire. Ce n'est pas ce que nous avons observé lors de nos mesures, le temps d'entraînement du jeu de données `15k_ga_adaptive` est plus élevé que celui de `30k_ga_adaptive` pre-filtrage, qui pourtant possède deux fois plus de données à partitionner.

4.4 Discussion

L'implémentation de **l'arbre de décision** pour la régression montre que la **qualité** des données est primordiale. Le filtrage des performances anormales permet de réduire significativement l'erreur et d'améliorer la stabilité du modèle. De plus, l'arbre de décision constitue une base solide pour les techniques ensemblistes telles que le bagging et le boosting, qui pourront être explorées dans les étapes suivantes du projet.

5 Analyse de la Vectorisation (SIMD)

Pour confirmer l'efficacité de la compilation avec les flags `-O3 -march=native`, nous avons inspecté le code machine généré pour la méthode `DecisionTreeRegressor::fit`.

L'objectif est de valider l'utilisation des registres **YMM** (256-bits) qui permettent de traiter 4 nombres flottants (doubles) simultanément (Single Instruction, Multiple Data), contre un seul en mode scalaire.

5.1 Inspection du Code Assembleur

La commande `objdump` révèle la séquence d'instructions suivante au cœur de la boucle de calcul du MSE :

```
726:  vbroadcastsd %xmm2,%ymm6      ; Duplication de la moyenne (mu) sur tout le registre
740:  vmovupd      (%rax),%ymm7     ; Chargement de 4 valeurs (x_i) depuis la mémoire
```

```
748:  vsubpd       %ymm6,%ymm7,%ymm1 ; Soustraction vectorielle : (x_i - mu)
74c:  vmulpd       %ymm1,%ymm1,%ymm1 ; Mise au carré vectorielle : (x_i - mu)^2
```

Interprétation Technique [7] :

— **Data Parallelism :** L'instruction `vsubpd` (Vector Subtract Packed Double) effectue 4 soustractions en un seul cycle d'horloge.

— **Reduction :** L'instruction `vbroadcastsd` montre que le compilateur a intelligemment détecté que la moyenne μ était une constante dans la boucle interne et l'a dupliquée pour la vectorisation.

— **Gain Théorique :** Sur des opérations arithmétiques pures, ce code est théoriquement 4 fois plus rapide qu'une version scalaire (SISD).

Cette analyse confirme que le compilateur GCC 13.3 a réussi à *auto-vectoriser* les boucles critiques de calcul de variance, exploitant pleinement le jeu d'instructions AVX2.

6 Optimisation

6.1 Optimisation Algorithmique : Formule de Koenig-Huygens

L'implémentation naïve du calcul du **MSE** nécessite de parcourir toutes les données pour calculer la moyenne, puis de repasser pour calculer la somme des carrés des écarts. Cela induit une complexité locale en $O(N^2)$, ce qui est prohibitif pour des jeux de donnéess de 15 000 et 30 000 lignes (temps initial observé : $\approx 18s$).

Pour optimiser cela, nous avons utilisé la propriété algébrique de la variance (formule de Koenig-Huygens) :

$$\sum (y_i - \bar{y})^2 = \sum y_i^2 - \frac{(\sum y_i)^2}{N} \tag{5}$$

Implémentation optimisée : En pré-calculant la somme cumulative des y_i et des y_i^2 lors du partitionnement des données, nous pouvons mettre à jour les statistiques des ensembles gauche et droite en temps constant $O(1)$ lors du déplacement du seuil de coupure.

— $Sum_L \leftarrow Sum_L + y_i$

— $SumSq_L \leftarrow SumSq_L + y_i^2$

— $MSE_L = \frac{SumSq_L}{N_L} - \left(\frac{Sum_L}{N_L}\right)^2$

Grâce à cette optimisation, la complexité de recherche du meilleur split passe de $O(N^2)$ à $O(N \log N)$ (coût du partitionnement). **Résultat :** Le temps d'entraînement est passé de 18 secondes à **0.4 seconde**.

7 Approche Ensembliste : Bagging et Boosting

Les méthodes ensemblistes reposent sur la combinaison de plusieurs modèles de base pour obtenir une prédiction plus robuste. Cette section introduit la distinction fondamentale entre deux types d'apprenants [8] :

— **Apprenants Forts (Strong Learners) :** Modèles complexes (ex : arbres profonds) ayant un faible biais mais une forte variance (tendance au sur-apprentissage).

— **Apprenants Faibles (Weak Learners) :** Modèles simples (ex : arbres peu profonds, "stumps") ayant un fort biais (ne capturent pas tout) mais une faible variance.

7.1 Le Bagging (Bootstrap Aggregating)

7.1.1 Concept Théorique

Le Bagging vise à réduire la **variance** d'un estimateur. Il utilise le principe de moyennage sur des modèles indépendants.

La prédiction finale est la moyenne de T modèles $\{h_1, ..., h_T\}$:

$$\hat{y}_{bagging}(x) = \frac{1}{T} \sum_{t=1}^{T} h_t(x) \tag{6}$$

7.1.2 Implémentation et Choix des Hyperparamètres

Pour notre implémentation 'BaggingRegressor', nous avons fait les choix architecturaux suivants :

1. **Bootstrap :** Chaque arbre est entraîné sur un échantillon tiré avec remise de même taille que le jeu de donnée original (Ratio 1.0).
2. **Profondeur (Strong Learners) :** Nous utilisons des arbres profonds (`max_depth = 12`). *Justification :* L'objectif est d'utiliser des modèles à **faible biais**. Individuellement, ces arbres sont en sur-apprentissage (en cherchant à minimiser l'erreur pure, l'arbre va diviser l'espace jusqu'à isoler des échantillons uniques, capturant ainsi le bruit aléatoire du jeu de donnée plutôt que la tendance réelle. Si le modèle performe parfaitement sur les données d'entraînement mais échoue sur de nouvelles données, il y a surapprentissage). C'est précisément l'agrégation (étape de moyenne) qui réduire la variance globale sans augmenter le biais.
3. **Nombre d'estimateurs (T) :** Fixé à 15 . *Justification :* Théoriquement, plus T est grand, plus la variance diminue. C'est un compromis Performance / Precision.

7.2 Le Gradient Boosting

7.2.1 Concept Théorique

Contrairement au Bagging qui parallélise des apprenants forts, le Boosting est une méthode séquentielle qui construit un **Strong Learner** à partir d'une suite de **Weak Learners**. L'objectif est de réduire le **biais**.

Le modèle s'écrit comme une somme pondérée [9] :

$$F_M(x) = F_0(x) + \sum_{m=1}^{M} \eta \cdot h_m(x) \tag{7}$$

7.2.2 Lien Mathématique : Gradient et Résidu [9]

Nous utilisons la fonction de perte quadratique (MSE) : $L(y, F(x)) = \frac{1}{2}(y - F(x))^2$. Pour minimiser cette perte par descente de gradient dans l'espace des fonctions, nous calculons la dérivée par rapport à la prédiction actuelle :

$$-\frac{\partial L}{\partial F} = -\frac{\partial}{\partial F}\left[\frac{1}{2}(y - F)^2\right] = (y - F) \tag{8}$$

Ainsi, pour la perte L_2, le **résidu** $(y - F_{m-1}(x))$ est exactement le **gradient négatif**. Entraîner l'arbre suivant sur les résidus revient donc à effectuer un pas de gradient vers la solution optimale.

7.2.3 Algorithme Implémenté

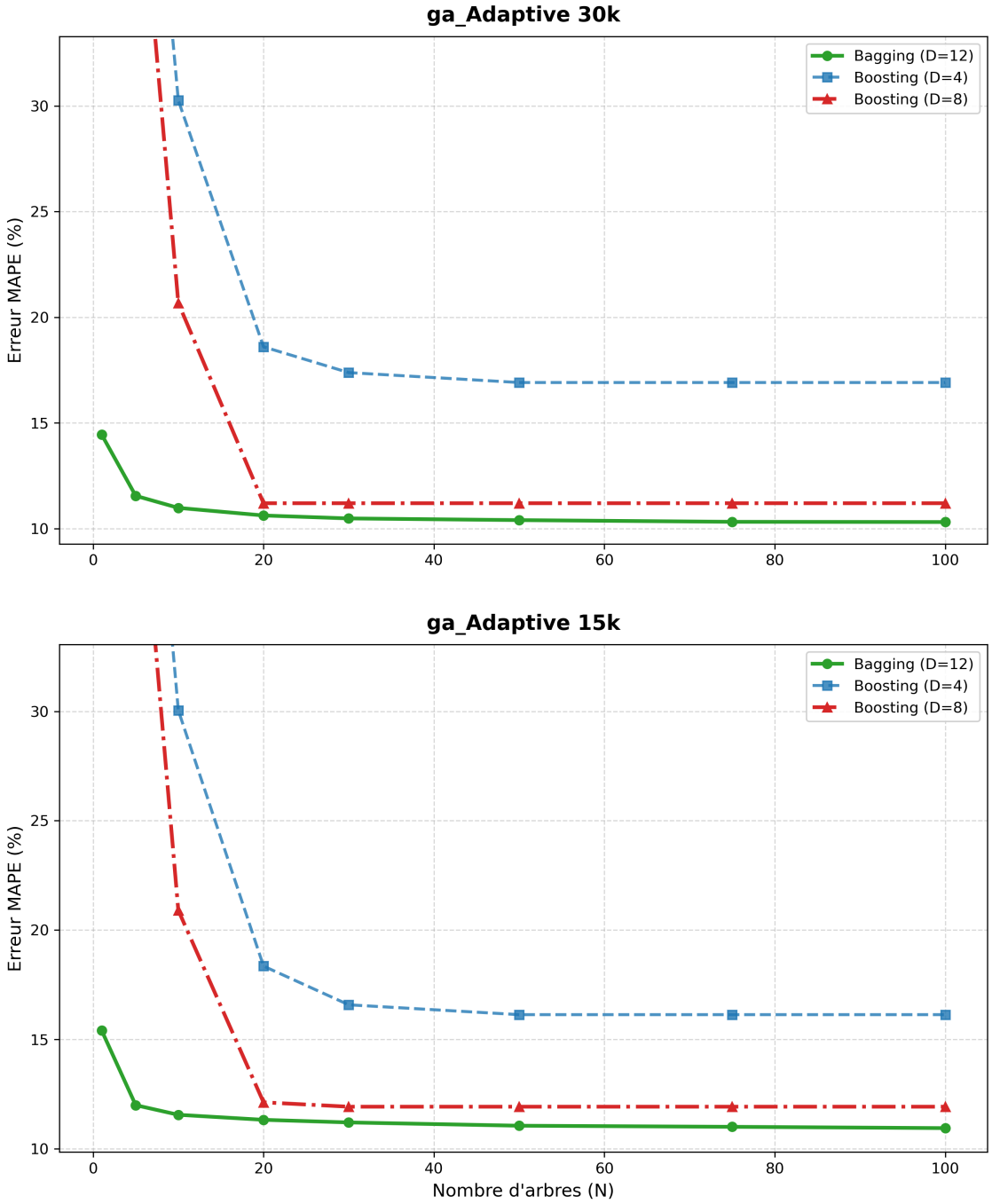
1. Initialisation : $F_0(x) = \text{moyenne}(y)$.
2. Pour $m = 1$ à M :
   (a) Calcul du gradient (résidu) : $r_{im} = y_i - F_{m-1}(x_i)$.
   (b) **Weak Learner :** Entraîner un arbre h_m sur $\{(x_i, r_{im})\}$.
   (c) *Justification de la profondeur (max_ depth=4) :* Contrairement au Bagging, nous devons utiliser des apprenants faibles (haut biais). Si l'arbre h_m était profond, il apprendrait parfaitement les résidus (y compris le bruit) dès la première itération, stoppant l'apprentissage. La faible profondeur force le modèle à n'apprendre que la structure dominante de l'erreur, laissant les détails aux itérations suivantes.
   (d) Mise à jour avec freinage : $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$.

8 Étude Paramétrique

Plutôt que de comparer les modèles sur un point d'arrêt arbitraire (ex : $N = 50$), nous avons mené une étude paramétrique complète (de $N = 1$ à $N = 100$ estimateurs) pour analyser la dynamique de convergence et le compromis Temps/Précision sur l'ensemble des jeux de données.

8.1 Précision

Les Figures synthétise l'évolution de l'erreur (MAPE) en fonction du nombre d'arbres pour trois configurations : le Bagging (Profondeur 12), le Boosting faible ($D = 4$) et le Boosting moyen ($D = 8$).



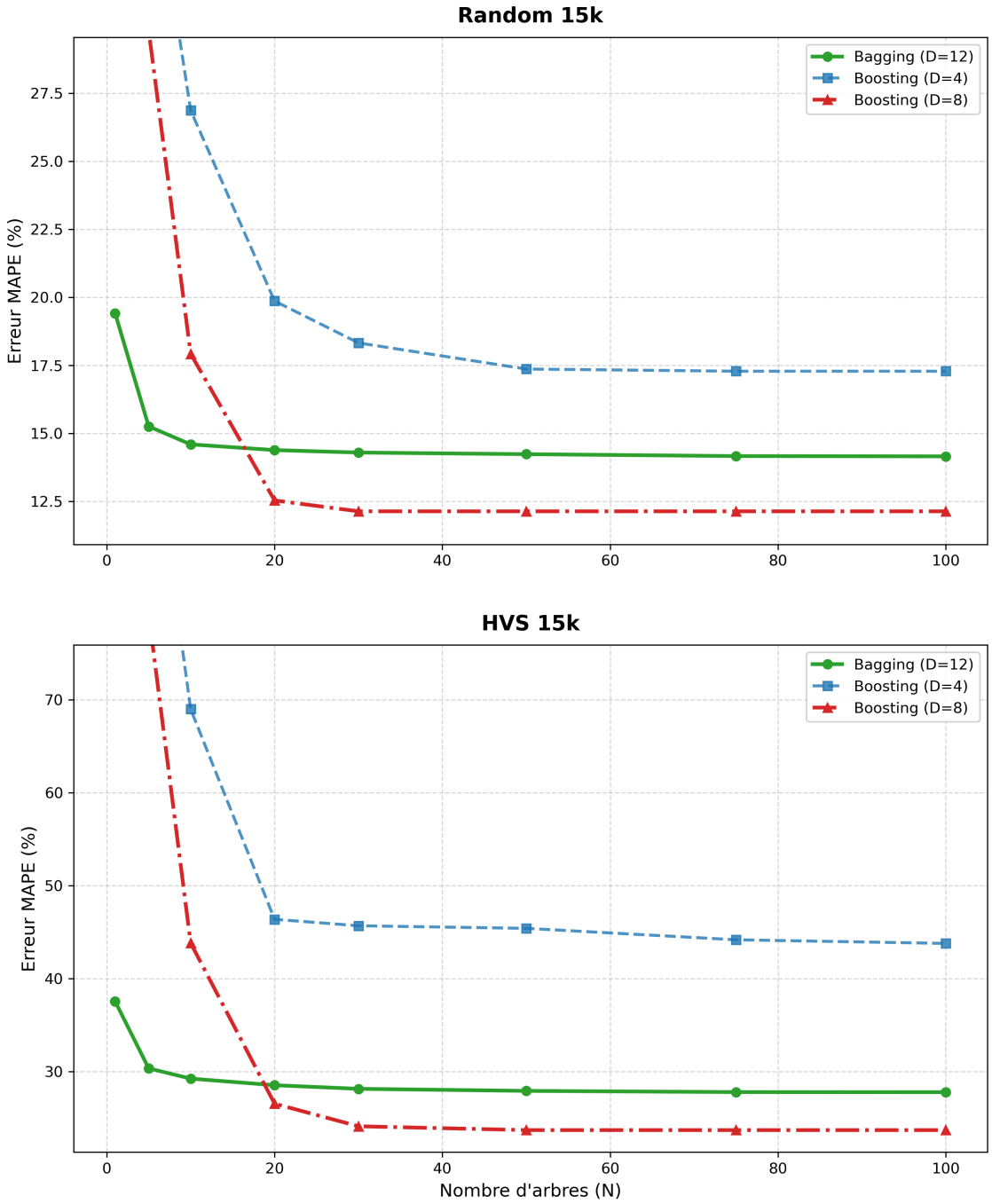


FIGURE 4 – Dynamique de convergence du MAPE sur les 4 jeux de données. Comparaison entre Bagging ($D = 12$) et Boosting ($D = 4$ vs $D = 8$).

Analyse des résultats :

— **Stabilité du Bagging (Vert) :** Sur tous les scénarios, le Bagging ($D = 12$) présente une courbe légèrement décroissante. L'erreur diminue rapidement jusqu'à $N = 30$, puis se stabilise. C'est la méthode la plus fiable "par défaut".

— **Limites du Boosting Faible (Bleu, $D = 4$) :** Sur les jeux de données complexes comme `15k_hvs` et `15k_random`, les apprenants de profondeur 4 souffrent d'un **biais élevé** . Même avec 100 arbres, ils ne parviennent pas à descendre sous les 16% d'erreur. contrairement au `30k_ga_adaptive` et `15k_ga_adaptive`, et la aussi le Bagging atteint 10.3%.

— **Performance du Boosting Moyen (Rouge, $D = 8$) :** C'est la révélation de cette étude. En

augmentant la capacité des apprenants faibles ($D = 8$), le Boosting converge extrêmement vite.

— **Cas 15k_hvs :** Sur ce jeu de donnée, le Boosting $D = 8$ surpasse le Bagging (MAPE **23.6%** contre 27.7̄%). Cela prouve que pour des topologies complexes, des apprenants plus "intelligents" sont nécessaires pour réduire le biais.

— **Dataset random, Boosting (Rouge, $D = 8$) :**

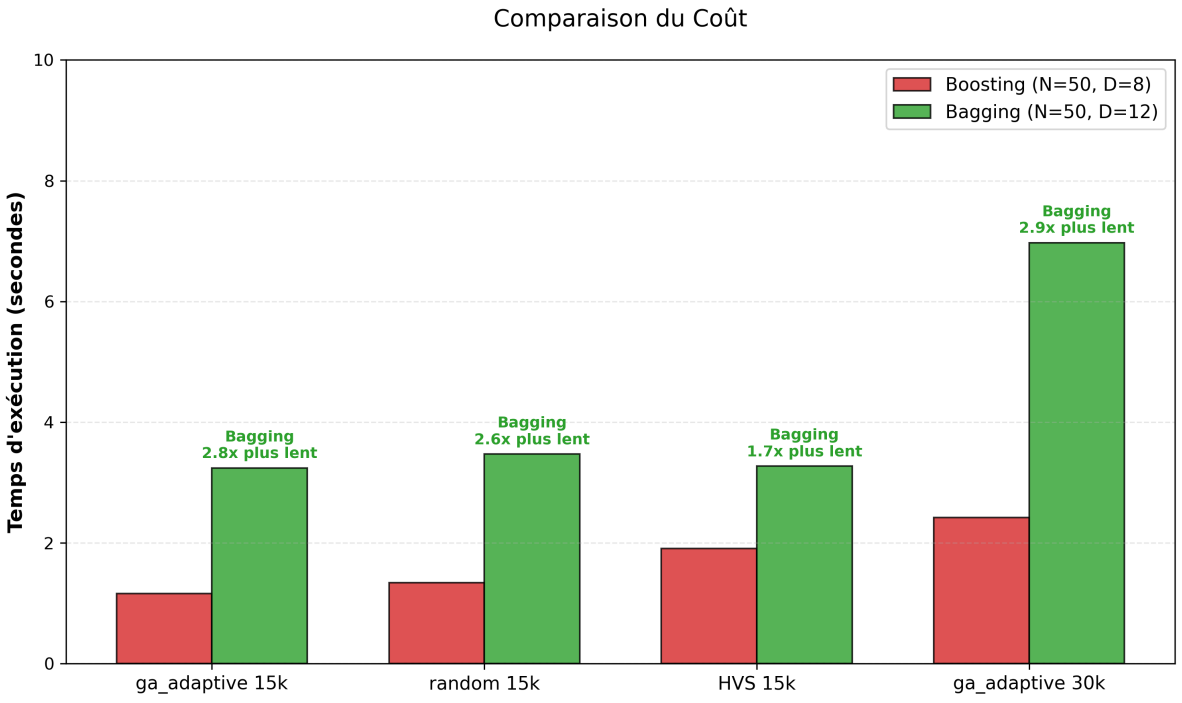
On remarque que sur ce jeu de données, tout les records précèdemment établis sont battus, du fait qu'il contient des configurations choisies aleatoirement contrairement au `15k_hvs`, `15k_ga_adaptive` et `30k_ga_adaptive` qui biaise le jeu de donnée.

8.2 Analyse du Coût Computationnel et Scaling

Le tableau ci-dessous compare les temps d'exécution pour $N = 50$ arbres, point où la majorité des modèles ont atteint leur plateau de convergence.

Dataset	Boosting ($N = 50, D = 8$)	Bagging ($N = 50, D = 12$)	Ratio Vitesse
15k_ga_adaptive	1.16 s	3.24 s	Boosting ×2.8
30k_ga_adaptive	**2.42 s**	6.97 s	Boosting ×2.9
15k_HVS	1.91 s	3.27 s	Boosting ×1.7
15_Random	1.34 s	3.47 s	Boosting ×2.58

TABLE 4 – Temps d'entraînement séquentiel pour 50 estimateurs.



Interprétation :

1. **Efficacité Séquentielle :** Le Boosting est systématiquement plus rapide (de ×1.7 à ×2.9). Cela s'explique par la complexité combinatoire des arbres : construire 50 arbres de profondeur 8 (Boosting) implique de visiter beaucoup moins de nœuds que 50 arbres de profondeur 12 (Bagging), car le nombre de feuilles croît exponentiellement avec la profondeur (2^D).
2. **Scaling Algorithmique (Passage à l'échelle) :** En comparant *15k_ga_adaptive* et *30k_ga_adaptive* pour le Bagging :

$$\frac{T_{30k}}{T_{15k}} = \frac{6.97s}{3.24s} \approx 2.15 \tag{9}$$

Pour un doublement de la charge ($N \times 2$), le temps augmente d'un facteur 2.15. Ce comportement quasi-linéaire ($O(N \log N)$ théorique) confirme la qualité de l'implémentation mémoire et l'absence de goulots d'étranglement quadratiques.

9 Étude d'Optimisation Bas-Niveau : Impact du Compilateur

Afin de garantir que nos mesures de performance reflètent les capacités réelles du matériel (Ryzen 7 7735HS) et non les limitations de la génération de code, nous avons confronté le compilateur standard GCC à ses concurrents basés sur l'infrastructure LLVM.

9.1 Protocole Expérimental

Nous avons comparé quatre chaînes de compilation modernes :
— **GCC 13.3.0** : La référence sous Linux Mint 22.
— **Clang 17.0.6** : Le compilateur du projet LLVM.
— **Intel oneAPI (ICX 2024.0)** : Réputé pour son autovectorisation agressive.
— **AMD AOCC 5.0** : Version optimisée par AMD pour l'architecture Zen.

Flags de Compilation : Pour assurer une comparaison équitable, tous les exécutables ont été compilés avec les options d'optimisation maximale et l'activation du jeu d'instructions natif de la machine :

```
-O3 -march=native -DNDEBUG
```

L'option `-march=native` permet aux compilateurs d'utiliser les instructions spécifiques AVX2, FMA3 et BMI2 supportées par le processeur.

10 Analyse d'Optimisation Bas-Niveau et Memory Wall

Cette section croise l'impact du compilateur avec l'analyse des compteurs de performance matériels (IPC - Instructions Per Cycle) obtenus via l'outil de profilage `perf`. L'objectif est de distinguer la charge de calcul pure (Compute Bound) des limitations matérielles d'accès aux données (Memory Bound) .

10.1 Analyse par Jeu de Données

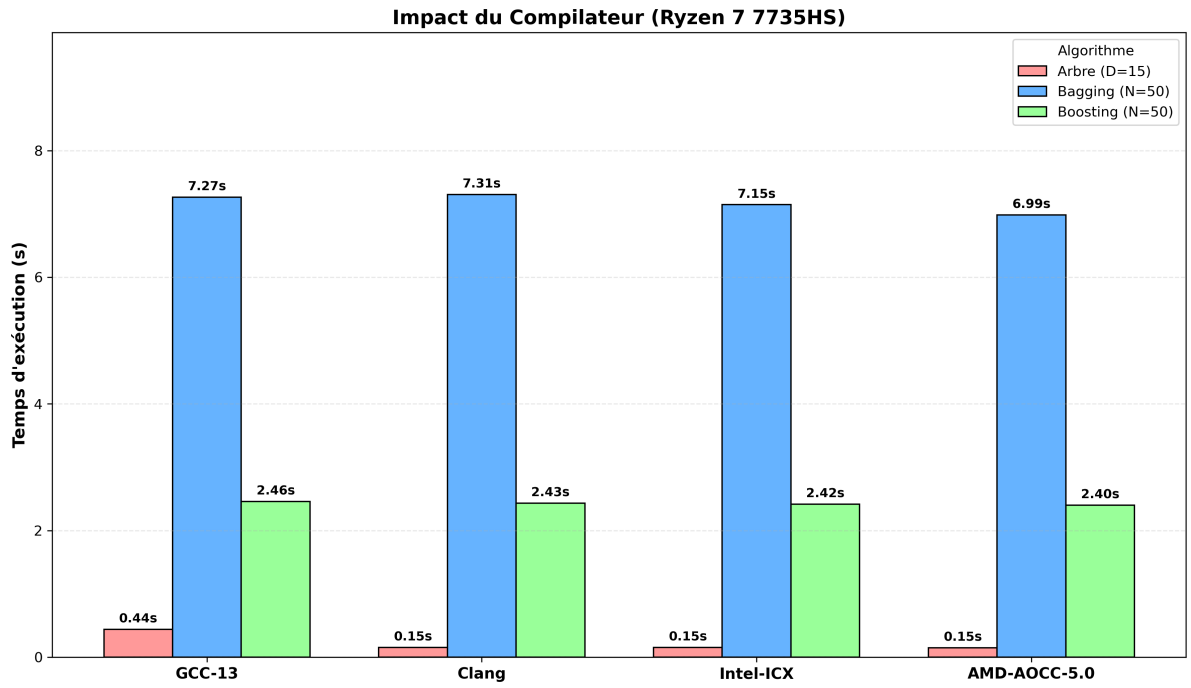


FIGURE 5 – Performance Compilateurs - Dataset `30k_ga_adaptive`.

Inspection des Résultats (`30k_ga_adaptive`) :

- **Impact Compilateur (Structure de Contrôle) :** Sur l'arbre unique ($D = 15$), nous observons un gain de facteur **3.1** en faveur des compilateurs LLVM (0.15s) contre GCC (0.47s). Cela valide la supériorité de l'optimisation *Tail Call* de Clang/Intel sur la récursion profonde.
- **Preuve du Memory Wall (Bagging) :** Les temps du Bagging stagnent autour de **7.2s**. L'analyse `perf` révèle un **IPC de 0.79**.
  - *Calcul du Working Set :* 30 000 lignes $\times$ 10 features $\times$ 8 octets $\approx$ 2.4 MB.
  - Pour 50 arbres, le volume de données actives dépasse 120 MB.
  - **Conclusion :** 120 MB $\gg$ 16 MB (L3 Cache). Le CPU est freiné par l'attente des données (IPC < 1.0), rendant l'optimisation du compilateur secondaire face à la latence RAM.

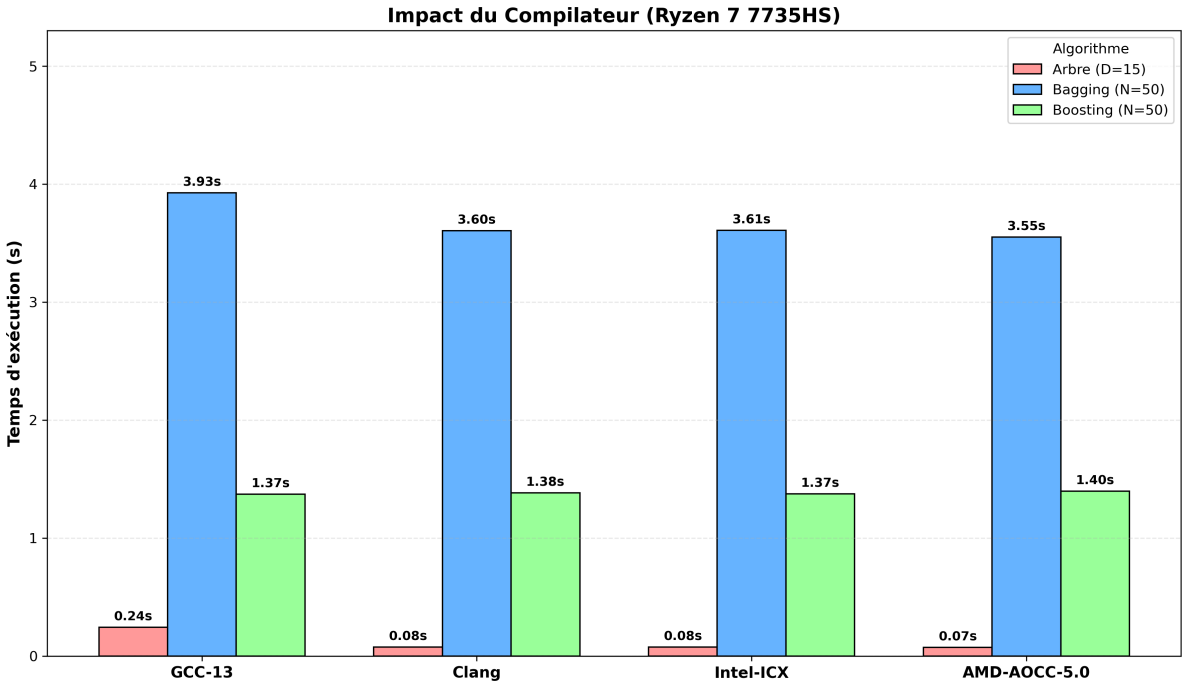


FIGURE 6 – Performance Compilateurs - Dataset `15k_random`.

Inspection des Résultats (`15k_random`) : L'IPC chute ici à **0.76**. .

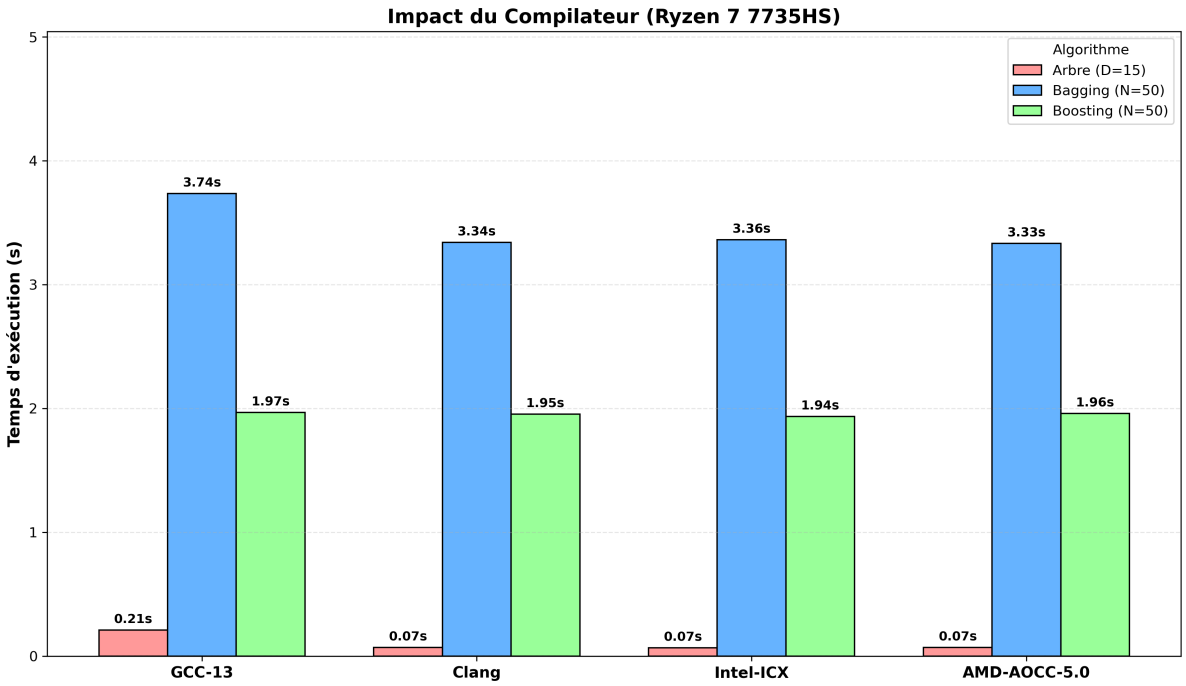


FIGURE 7 – Performance Compilateurs - Dataset `15k_hvs`.

Inspection des Résultats (15k_hvs) : Ce graphique illustre la scalabilité linéaire de notre implémentation.

- **Scalabilité :** Avec 15 000 lignes (soit la moitié du jeu de donnée `30k`), le temps d'exécution tombe à environ **3.35s** (contre 7.2s). Le temps est donc divisé par deux, suivant exactement la réduction du volume de données ($O(N)$).
- **Cohérence :** Malgré un changement de jeu de données, la hiérarchie des compilateurs reste identique : Clang, Intel et AOCC sont coude-à-coude et devancent GCC, confirmant que cette performance est structurelle.

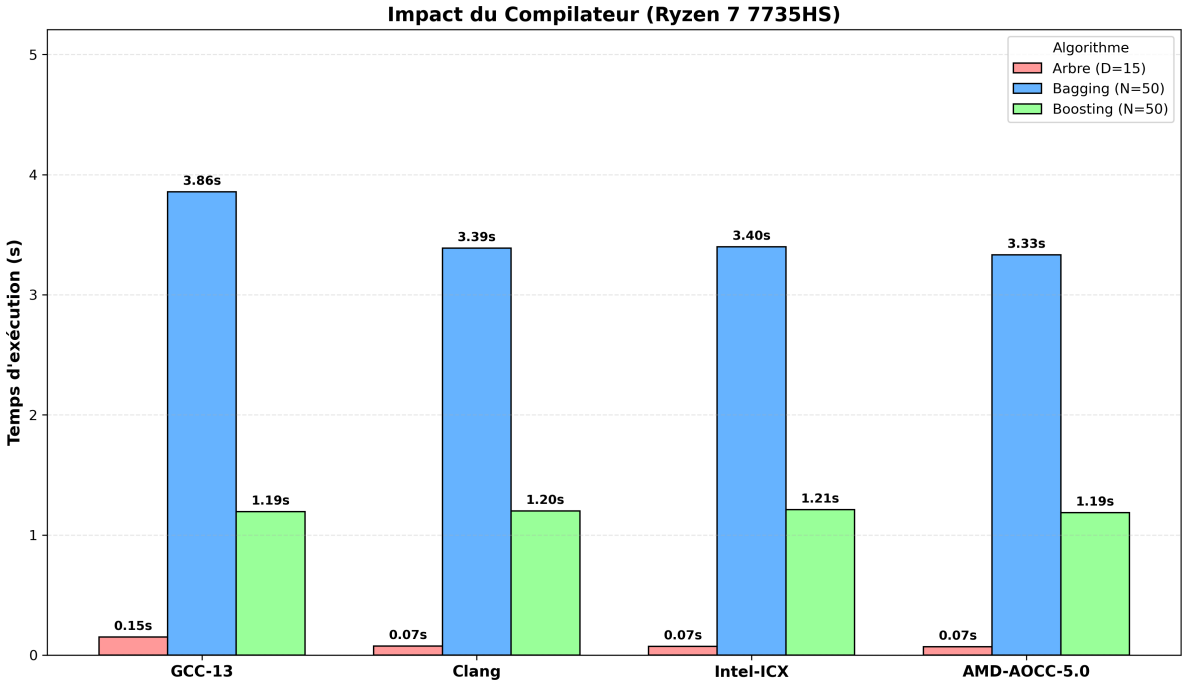


FIGURE 8 – Performance Compilateurs - Dataset `15k_ga_adaptive`.

Inspection des Résultats (15k_ga_adaptive) : Les résultats sont quasi-identiques à ceux du `15k_hvs` (3.4s), avec un IPC remontant légèrement à **0.80**.

11 Conclusion

Ce projet a permis d'étudier de manière approfondie les méthodes ensemblistes appliquées à un problème concret de modélisation de performances, en partant d'une implémentation d'arbre de décision pour la régression jusqu'à l'intégration du bagging et du boosting. Le choix d'une implémentation *from scratch* en **C++** a constitué un point central du travail, en imposant une réflexion détaillée sur les critères de division, les règles d'arrêt, la gestion des données et les aspects de performance liés à l'architecture matérielle.

Les expériences menées mettent en évidence que la qualité des données joue un rôle déterminant dans le comportement des modèles. Le prétraitement, et en particulier la gestion des valeurs aberrantes issues d'exécutions erronées, conditionne directement la stabilité de l'apprentissage et la pertinence des prédictions. Cette observation souligne que, dans un contexte de modélisation de performances, les choix effectués en amont de l'apprentissage sont aussi importants que le modèle lui-même.

L'analyse comparative entre un arbre de régression seul et les méthodes ensemblistes montre que l'agrégation de modèles permet d'améliorer la robustesse des prédictions, mais au prix d'un coût de calcul accru. Le bagging apparaît comme une approche stable et relativement simple à paramétrer, tandis que le boosting se révèle plus sensible aux réglages, notamment à la capacité des arbres utilisés. Ces résultats illustrent les compromis classiques entre biais, variance et complexité algorithmique, tels qu'ils se manifestent concrètement dans notre cadre expérimental.

Enfin, l'étude des aspects bas-niveau (optimisation du calcul du critère, vectorisation automatique, impact du compilateur et des accès mémoire) met en évidence que les performances observées ne dépendent pas uniquement des choix algorithmiques, mais également des contraintes matérielles. En particulier, lorsque le volume de données manipulé devient important, les limitations liées à la hiérarchie mémoire influencent fortement le temps d'exécution, indépendamment du compilateur utilisé.

En conclusion, ce travail a permis de mettre en place une implémentation de référence des méthodes ensemblistes pour la régression, ainsi qu'un protocole d'évaluation adapté à la modélisation de performances logicielles. Cette base sera exploitée au second semestre pour la parallélisation de la construction des arbres, l'étude de méthodes ensemblistes distribuées et la comparaison de l'impact des optimisations sur les performances et la précision des modèles.

Références

[1] Gavin Hackeling. *Mastering Machine Learning with Scikit-Learn*. Packt Publishing, 2014.

[2] J. R. Quinlan. « Induction of Decision Trees ». In : *Machine Learning* 1.1 (1986), p. 81-106. doi : 10.1007/BF00116251.

[3] Shan Suthaharan. *Machine Learning Models and Algorithms for Big Data Classification*. T. 36. Integrated Series in Information Systems. Springer US, 2016. doi : 10.1007/978-1-4899-7641-3.

[4] Tianfeng Chai et Ronald R. Draxler. « Root mean square error (RMSE) or mean absolute error (MAE) ? » In : *Geoscientific Model Development Discussions* 7 (2014), p. 1525-1534.

[5] Sebastian Raschka. *STAT 451 : Machine Learning Lecture Notes*. 2020.

[6] F. Pedregosa et al. « Scikit-learn : Machine Learning in Python ». In : *Journal of Machine Learning Research* 12 (2011), p. 2825-2830.

[7] Félix Cloutier. *x86 and amd64 instruction reference*. 2024. url : https://www.felixcloutier.com/x86/.

[8] Harris Drucker et Corinna Cortes. « Boosting Decision Trees ». In : *Advances in Neural Information Processing Systems*. Sous la dir. de D. Touretzky, M.C. Mozer et M. Hasselmo. T. 8. MIT Press, 1995. url : https://proceedings.neurips.cc/paper_files/paper/1995/file/4a08142c38dbe374195d41c04562d9f8-Paper.pdf.

[9] Jerome H. Friedman. « Greedy Function Approximation : A Gradient Boosting Machine ». In : *Annals of Statistics* 29.5 (2001), p. 1189-1232. doi : 10.1214/aos/1013203451.

[10] Dimitris Bertsimas et Berk Öztürk. « Global Optimization via Optimal Decision Trees ». In : *Journal of Global Optimization* (2023). doi : 10.1007/s10898-023-01311-x.

[11] Stéphane Caron. *Une introduction aux arbres de décision*. n.d.

[12] A. C. Davison et S. Sardy. *Méthodes de rééchantillonnage pour l'estimation de variance en sondage*. n.d.

[13] Yannig Goude. *Arbres de Régression, CART*. n.d.

[14] Guolin Ke et al. *LightGBM : A Highly Efficient Gradient Boosting Decision Tree*. n.d.

[15] Han Liu et Mihaela Cocea. « Semi-Random Partitioning of Data into Training and Test Sets in Granular Computing Context ». In : *Granular Computing* 2.4 (2017), p. 357-386. doi : 10.1007/s41066-017-0049-2.

[16] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. n.d.

[17] Bernard M. E. Moret. « Decision Trees and Diagrams ». In : *ACM Computing Surveys* 14.4 (1982), p. 593-623. doi : 10.1145/356893.356898.

[18] S. K. Murthy, S. Kasif et S. Salzberg. « A System for Induction of Oblique Decision Trees ». In : *Journal of Artificial Intelligence Research* 2 (1994), p. 1-32. doi : 10.1613/jair.63.

[19] J. P. Nakache. *Méthode de discrimination basée sur la construction d'un arbre de décision binaire*. n.d.

[20] Mehrbakhsh Nilashi, Rabab Ali Abumalloh, Behrouz Minaei-Bidgoli et al. « Predicting Parkinson's Disease Progression : Evaluation of Ensemble Methods in Machine Learning ». In : *Journal of Healthcare Engineering* (2022), p. 1-17. doi : 10.1155/2022/2793361.

[21] Paweł Piotrowski et al. « Advanced Ensemble Methods Using Machine Learning and Deep Learning for One-Day-Ahead Forecasts of Electric Energy Production in Wind Farms ». In : *Energies* 15.4 (2022), p. 1252. doi : 10.3390/en15041252.

[22] J. Ross Quinlan. *C4.5 : Programs for Machine Learning*. Morgan Kaufmann, 2014.

[23] J. Ross Quinlan. *Inferring Decision Trees Using the Minimum Description Length Principle*. n.d.

[24] Yan-yan Song et Ying Lu. « Decision Tree Methods : Applications for Classification and Prediction ». In : *Statistics* 27.2 (2015).

[25] Sami Tibshirani, Harry Friedman et Patrick Hastie Valerie. *s.d.* n.d.

[26] Bianca Zadrozny et Charles Elkan. *Obtaining Calibrated Probability Estimates from Decision Trees and Naive Bayesian Classifiers*. n.d.

[27] Thomas G. Dietterich. « Ensemble Methods in Machine Learning ». In : *Multiple Classifier Systems*. Springer, 2000, p. 1-15. DOI : 10.1007/3-540-45014-9_1.

[28] Leo Breiman. « Bagging Predictors ». In : *Machine Learning* 24.2 (1996), p. 123-140. DOI : 10.1007/BF00058655.

[29] Leo Breiman. « Random Forests ». In : *Machine Learning* 45.1 (2001), p. 5-32. DOI : 10.1023/A:1010933404324.

[30] Tianqi Chen et Carlos Guestrin. « XGBoost : A Scalable Tree Boosting System ». In : *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. 2016, p. 785-794. DOI : 10.1145/2939672.2939785.

[31] Guolin Ke et al. « LightGBM : A Highly Efficient Gradient Boosting Decision Tree ». In : *Advances in Neural Information Processing Systems (NeurIPS)*. NeurIPS. 2017. URL : https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bb9d9e6b76fa-Abstract.html.

[32] Michael J. A. Berry et Gordon S. Linoff. *Mastering Data Mining : The Art and Science of Customer Relationship Management*. 1st. Wiley, 1999. ISBN : 978-0471246303.

[33] Hosameldin Ahmed et Asoke K. Nandi. *Condition Monitoring with Vibration Signals : Advanced Time, Frequency, Time-Frequency, and Machine Learning Techniques*. Wiley-IEEE Press, 2020. ISBN : 978-1-119-54462-2.

[34] Pierre Geurts, Damien Ernst et Louis Wehenkel. « Extremely Randomized Trees ». In : *Machine Learning* 63.1 (2006), p. 3-42. DOI : 10.1007/s10994-006-6226-1.

[35] Yoav Freund et Robert E. Schapire. « A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting ». In : *Journal of Computer and System Sciences* 55.1 (1997), p. 119-139. DOI : 10.1006/jcss.1997.1504.

[36] Analytics Vidhya. *How to Implement the AdaBoost Algorithm ?* Accessed : 19 décembre 2025. 2021. URL : https://www.analyticsvidhya.com/blog/2021/09/adaboost-algorithm-a-complete-guide-for-beginners/.

[37] Data Science Wizards. *Understanding the AdaBoost Algorithm*. Accessed : 19 décembre 2025. 2023. URL : https://medium.com/@datasciencewizards/understanding-the-adaboost-algorithm-2e9344a833d9.

[38] Lars Buitinck et al. « API design for machine learning software : experiences from the scikit-learn project ». In : *ECML PKDD Workshop : Languages for Data Mining and Machine Learning*. 2013, p. 108-122.

[39] Wm A Wulf et Sally A McKee. « Hitting the memory wall : implications of the obvious ». In : *ACM SIGARCH computer architecture news* 23.1 (1995). Référence clé pour le goulot d'étranglement mémoire, p. 20-24.

[40] Advanced Micro Devices. *Software Optimization Guide for AMD Family 19h Processors*. Documentation technique officielle de l'architecture Zen 3 (Ryzen 7 7735HS). AMD. 2020.

[41] Chris Lattner et Vikram Adve. « LLVM : A compilation framework for lifelong program analysis & transformation ». In : *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* Architecture du compilateur Clang/LLVM. IEEE. 2004, p. 75-86.

[42] Intel Corporation. *Intel® oneAPI DPC++/C++ Compiler Developer Guide and Reference*. Référence pour le compilateur ICX et les optimisations vectorielles. 2024.

[43] Linux Kernel Organization. *perf : Linux profiling with performance counters*. Outil utilisé pour mesurer l'IPC et les cache misses. 2024. URL : https://perf.wiki.kernel.org/.

[44] Agner Fog. *Optimizing software in C++ : An optimization guide for Windows, Linux and Mac platforms*. La référence absolue pour l'optimisation bas-niveau sur x86. 2023. URL : https://www.agner.org/optimize/.

[45] John L Hennessy et David A Patterson. *Computer architecture : a quantitative approach*. Ouvrage de référence pour les concepts d'IPC et de hiérarchie mémoire. Elsevier, 2011.